

Assesment-5 Car

Aim: To write verilog code for SR, JK, D, T, flip flop & verify it functionally

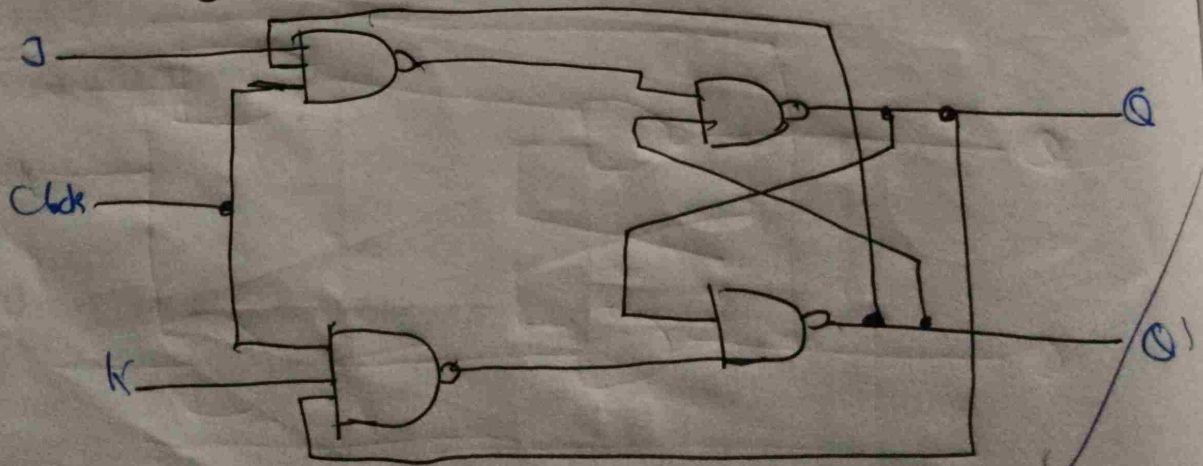
Tools Required: VSI

@Theory: SR FLIP a two-state memory circuit with set (S) & reset (R) inputs that stores a single bit of info.

Truth table:

S	R	$Q(n+1)$	State
0	0	Q_n	No change
0	1	0	RESET
1	0	1	SET
1	1	X	Invalid

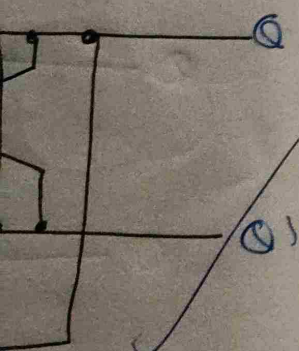
Circuit Diagram



Ca
JK, 0, 1, flip flop

clock with set (s)
forces a single

state
change
EST
EST
valid



Code

```
module sr_ff (clk, rst, s, r, q);
    input clk, rst, s, r;
    output reg q;
    always @(posedg clk or posedge rst)
    begin
        if (rst == 1)
            q <= 0;
        else if (s == 0 && r == 0)
            q <= q;
        else if (s == 0 && r == 1)
            q <= 0;
        else if (s == 1 && r == 0)
            q <= 1;
        else if (s == 1 && r == 1)
            q <= ~q;
    end
endmodule
```

```
module t6();
    reg clk, rst, s, r;
    wire q;
    sr_ff uut (clk, rst, s, r, q);
    always begin
        #10 clk = ~clk;
    end
    initial
```

Op verified
25-09-2025

begin

clk = 0; cnt = 0; r = 0; s = 1;

#5 cnt = 1;

#5 cnt = 0; r = 1; s = 0;

#20 s = 1; r = 0;

#20 s = 1; r = 1;

#20 s = 0; r = 0;

#20 cnt = 1;

#5 cnt = 0;

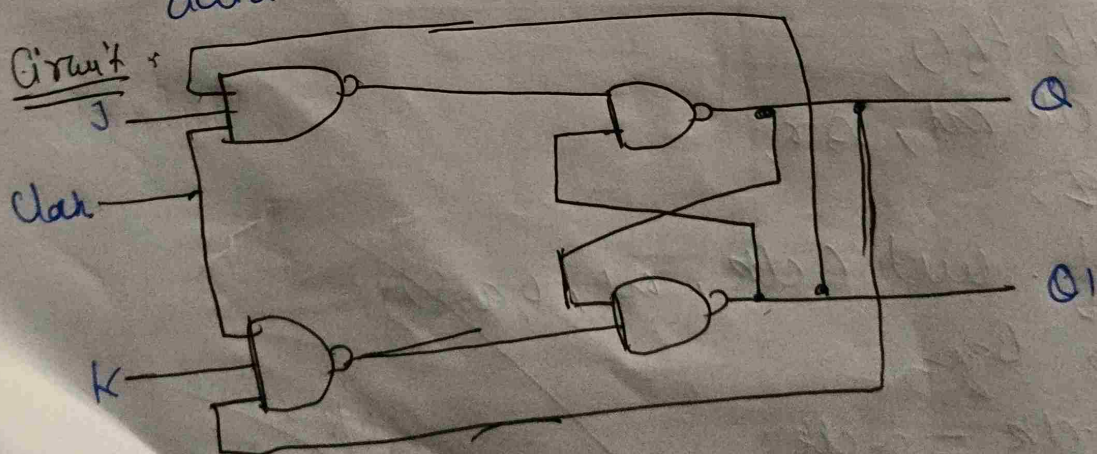
\$ stop;

end

endmodule

JK-Flip

theory: a type of electronic memory device that stores a single bit of information and can be used to manipulate digital data.



Truth table

J	K	Q(n+1)	State
0	0	Qn	No change
0	1	0	RESET
1	0	1	SET
1	1	Qn'	Toggle

Code

```

module jk_ff (clk, rst, j, k, q);

```

```

    input clk, rst, j, k;

```

```

    output reg q;

```

```

    always @ (posedge clk or posedge rst)
    begin

```

```

        if (rst == 1)

```

```

            q <= 0;

```

```

        else if (j == 0 && k == 0)

```

```

            q <= q;

```

```

        else if (j == 1 && k == 0)

```

```

            q <= 1;

```

```

        else if (j == 1 && k == 1)

```

```

            q <= ~q;

```

```

        end

```

```

    endmodule

```

*IP verified
 Rishy
 25-09-2025


```

module tb1;
reg clk, rst, j, k;
wire q;
jk_ff out (clk, rst, j, k, q);
always begin
# clk = 2 * clk;
end
initial
begin
clk = 0; rst = 0; j = 0; k = 0;
# 5 rst = 1;
# 5 rst = 0; j = 0; k = 0;
# 25 j = 0; k = 1;
# 25 j = 1; k = 1;
# 25 j = 1; k = 0;
# 5 rst = 1;
# 25 rst = 0; j = 1; k = 1;
$stop;
end
endmodule

```

theory:

Circuit

1-

u

~~25~~

For

m

i

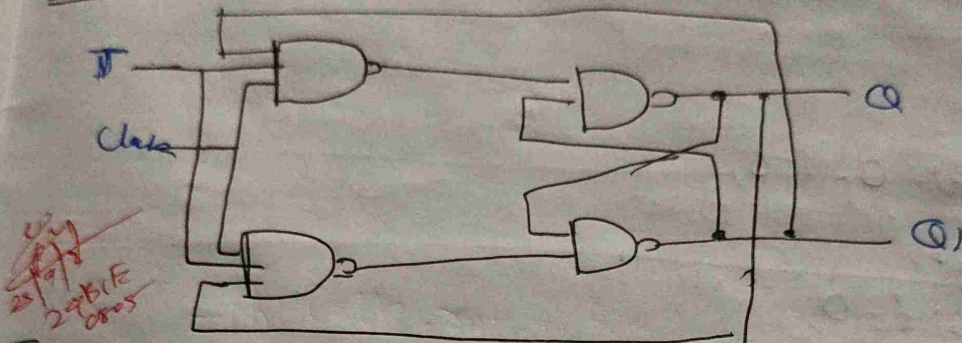
e

c

T flip flop

theory: a digital electronic circuit with a single input (T) that changes its output state (Q) when enabled. clock pulse with T to high and maintain its current state when T to low

Circuit



Truth table

T	Q(n+1)	State
0	Q	No change
1	Q'	Toggle

Code

```
module t_ff (clk, rst, t, q);  
  input clk, rst, t;  
  output reg q;  
  always @(posedge clk or posedge rst)  
  begin  
    if (rst == 1)  
      q <= 0;  
    else if (t == 1)  
      q <= ~q;  
    else  
      q <= q;  
  end  
end
```



```

module tb1;
  reg clk, rst, t;
  wire q;
  b-hw (clk, rst, bqr);
  always begin
    #10 clk = ~clk;
    end
  initial
    begin
      clk = 0; rst = 0; t = 0;
      #5 rst = 1;
      #10 t = 0;
      #15 t = 1;
      #20 rst = 1;
      #25 rst = 0; t = 1;
      $stop;
    end
endmodule

```

Theory: a
stor
a d
imp
d

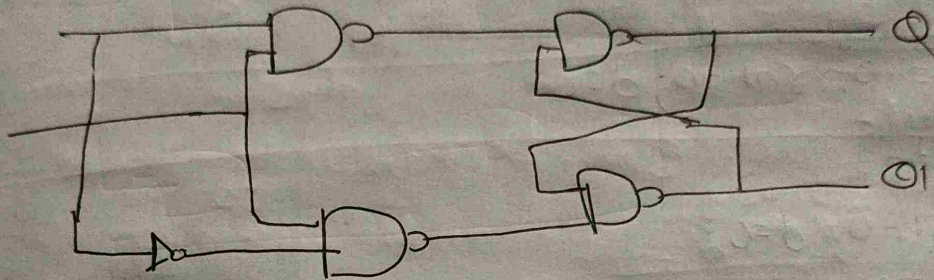
Circuit

Tr

D flip flop

Theory: a clock sequential logic circuit - that stores a single bit of binary data, acting as a delay element by copying its data (b) input to its output (Q) at the moment of a clock edge.

Circuit:



Truth-Table

D	Q (next)	State
0	0	RESET
1	1	SET

Code:

```
module dff (clk, rst, d, q);  
    input clk, rst, d;  
    output reg q;  
    always @ (posedge clk or posedge rst)  
    begin  
        if (rst == 1)  
            q <= 0;  
        else  
            q <= d; end endmodule
```

O/p Verified
Date
25-09-2025


```

module tb1;
  reg clk, rst, d;
  wire q;
  dff q1 (clk, rst, d, q);
  always begin
    #10 clk = ~clk;
  end
  initial
  begin
    #5 clk = 0, rst = 0, d = 0;
    #5 rst = 1;
    #5 rst = 0, d = 0;
    #15 d = 1;
    #10 d = 0;
    #15 d = 1;
    #5 rst = 1;
    #5 rst = 0, d = 1;
    $stop;
  end
endmodule

```


Assessment - S(ub)

Aim: To write the verilog Code for SISO
SISO shift register

Tools Required: VSIM

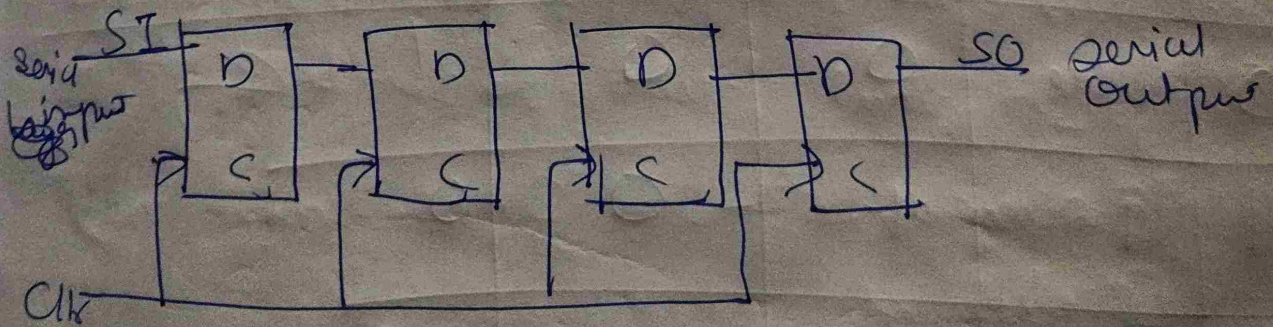
Theory:

SISO: fundamental sequential digital logic circuits that shift data bit-by-bit, controlled by a clock signal

SISO: a shift register also uses a series of interconnected flip-flops, but each flip-flop's output is accessible as a separate parallel output

Circuit Diagram

SISO



S2PO



Truth table:

S2PO

CLK	'Q3'	'Q2'	'Q1'	'Q0'
1st	0	0	0	0
2nd	1	0	0	0
3rd	1	1	0	0
4th	1	1	1	0

S2PO

Clock	Q1	Q2	Q3	Q4
0	1	0	0	1
1	1	1	0	0
2	0	1	1	0
3	0	0	1	1

Code

```
module SISO (o0, si, clk, rst);
```

```
output o0;
```

```
input si, clk, rst;
```

```
reg [3:0] q;
```

```
always @ (posedge clk or posedge rst)
```

```
begin
```

```
if (rst)
```

```
q <= 4'b0;
```

```
else
```

```
q <= {0, q[3:1]};
```

```
end
```

```
assign s = q[0]
```

```
endmodule
```

```
module student ()
```

```
begin
```

```
wire o0;
```

```
reg si, clk, rst;
```

```
si <= 0
```

```
o0 (<= o0, si, clk, rst);
```

```
always
```

```
# 5 clk = ~clk;
```

```
initial begin
```

```
si <= 0; clk <= 0; rst <= 1;
```

```
# 10 rst <= 0;
```

```
si <= 1;
```



```

# 10 q1 = 0;
# 10 q1 = 0;
# 10 q1 = 1;
# 10 q1 = 1;
# 50 $stop;
end
endmodule

```

Code

```

module sipo(q, si, clk, rst);
    output [3:0] q;
    input si, clk, rst;
    reg [3:0] q;
    always @ (posedg clk or posedge rst)
    begin
        if (rst)
            q <= 4'b0;
        else
            q <= {q1, q[3:1]};
    end
endmodule

```

```

module sipo_tb;
    wire [3:0] q;
    reg si, clk, rst;
    sipo el(q, si, clk, rst);
    always
    # 5 clk = ~clk;

```


initial
regin

ar = 0;

clk = 0;

out = 1;

#10 out = 0;

#100 = 1;

#10 ar = 0;

#10 ar = 0;

#10 ar = 1;

#10 ar = 0;

#10 ar = 1;

#50 \$stop;

end

endmodule

011
944
301111
248 000005 944

Conclusion

The verilog code is successfully unified

Assessment - 1st

Aim - To verify verilog code for up-down counter
(b) Module N-count (n=10)
(c) Johnson counter

Tools Required: Modelsim

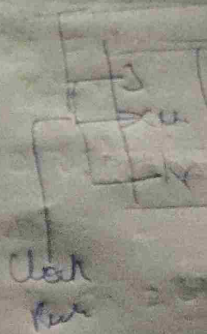
Theory - UP-down Counter: A counter that can go from 0 to count in either an ascending or descending order based on a control signal. It increases from a low value to a high value or decreases from a high value to a low value.

Module-n-Counter: A counter with a specific number of unique states, denoted by 'n'. It starts from 0 to $n-1$ & then resets, having a total of n unique states.

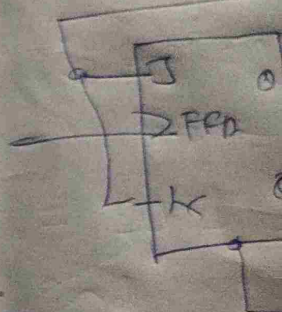
Johnson Counter: A counter where the inverted output of the last flip-flop is connected to the input of the first flip-flop. This creates a sequence of states that is twice the number of flip-flops. For example, a 3-stage Johnson counter has a modulus of 6.

Circuit Diagram

UP-down



Module-n counter

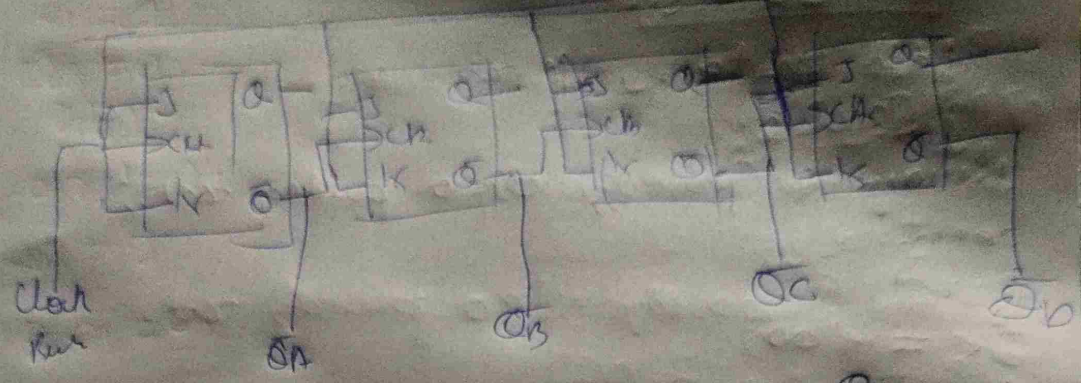


Johnson

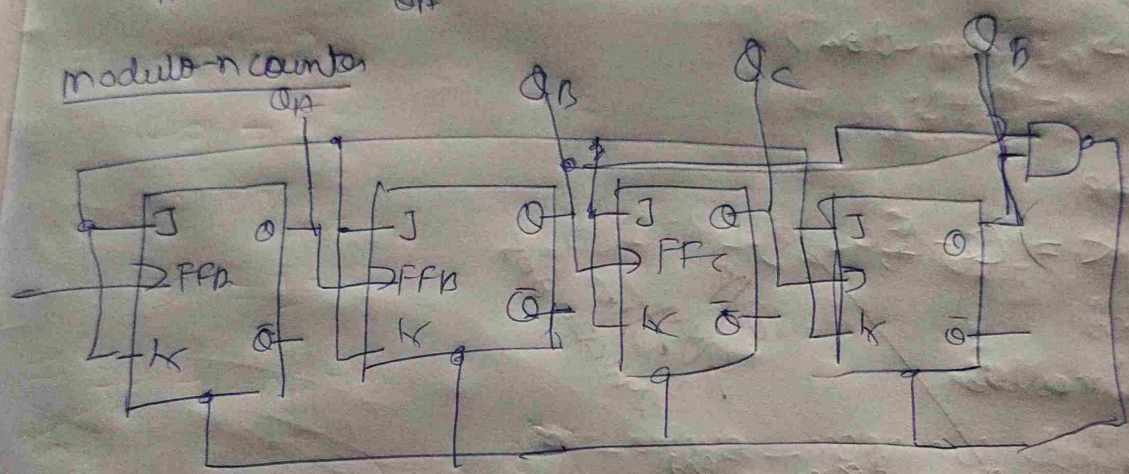


Circuit Diagram

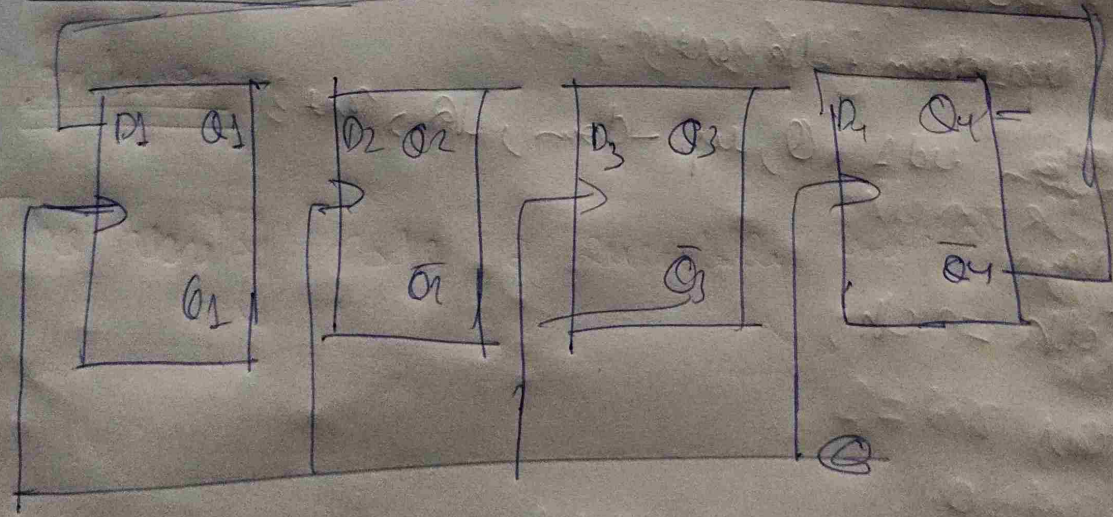
UP-down



modulo-n counter



Johnson Counter



Q code

```
module up-down (Q, up-down, clk, rst);  
    Output [3:0] Q;  
    input up-down, clk, rst;  
    reg [3:0] Q;  
    always @ (posedg clk or posdeg rst);  
    begin  
        if (rst)  
            Q <= 4'b0;  
        else if (up-down)  
            Q <= Q + 1;  
        else  
            Q <= Q - 1;  
        end  
    endmodule
```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

```
module up-down-test;  
    wire [3:0] Q;  
    reg up-down, clk, rst;  
    up-down ud1 (Q, up-down, clk, rst);  
    always  
        # 5 clk = ~clk;
```

initial

begin

(clk = 0) up-down = 1; rst = 1;

10 rst = 0;

100 up-down = 0;

200 \$stop;

end
endmodule

b) Code

```
module modN_counter (O, clk, rst);  
    output [3:0] O;  
    input  clk, rst;  
    reg [3:0] O;  
    always @(posedge clk or posedge rst);
```

```
begin  
    if (rst)  
        O <= 4'b0;  
    else if (O == 9)  
        O <= 4'b0;  
    else  
        O <= O + 1;  
    end  
endmodule
```

```
module modN_counter_tb;  
    wire [3:0] O;  
    reg clk, rst;  
    modN_counter mcb (O, clk, rst);  
    always
```

```
#5 clk = ~clk; //  
initial  
begin  
    clk = 0; rst = 1;  
#10 rst = 0;  
#200 $stop;
```

200ns
30ns
200ns 0.805

end
endmodule

(C) Johnson Counter

```
module johnson_counter (q, clk, reset);  
    output [3:0] q;  
    input clk, rst;  
    reg [3:0] q;  
    always @(posedg clk or posedge rst)
```

```
begin  
    if (rst)  
        q <= 4'b0000;  
    else  
        q <= {~q[0], q[3:1]};  
    end  
endmodule
```

O/P verified
Silly
30-10-2025

```
module johnson_counter_tb;
```

```
    wire [3:0] q;
```

```
    reg clk, rst;
```

```
    johnson_counter j1 (q, clk, rst);
```

```
    always
```

```
    # 5 clk = ~clk;
```

```
    initial
```

```
    begin
```

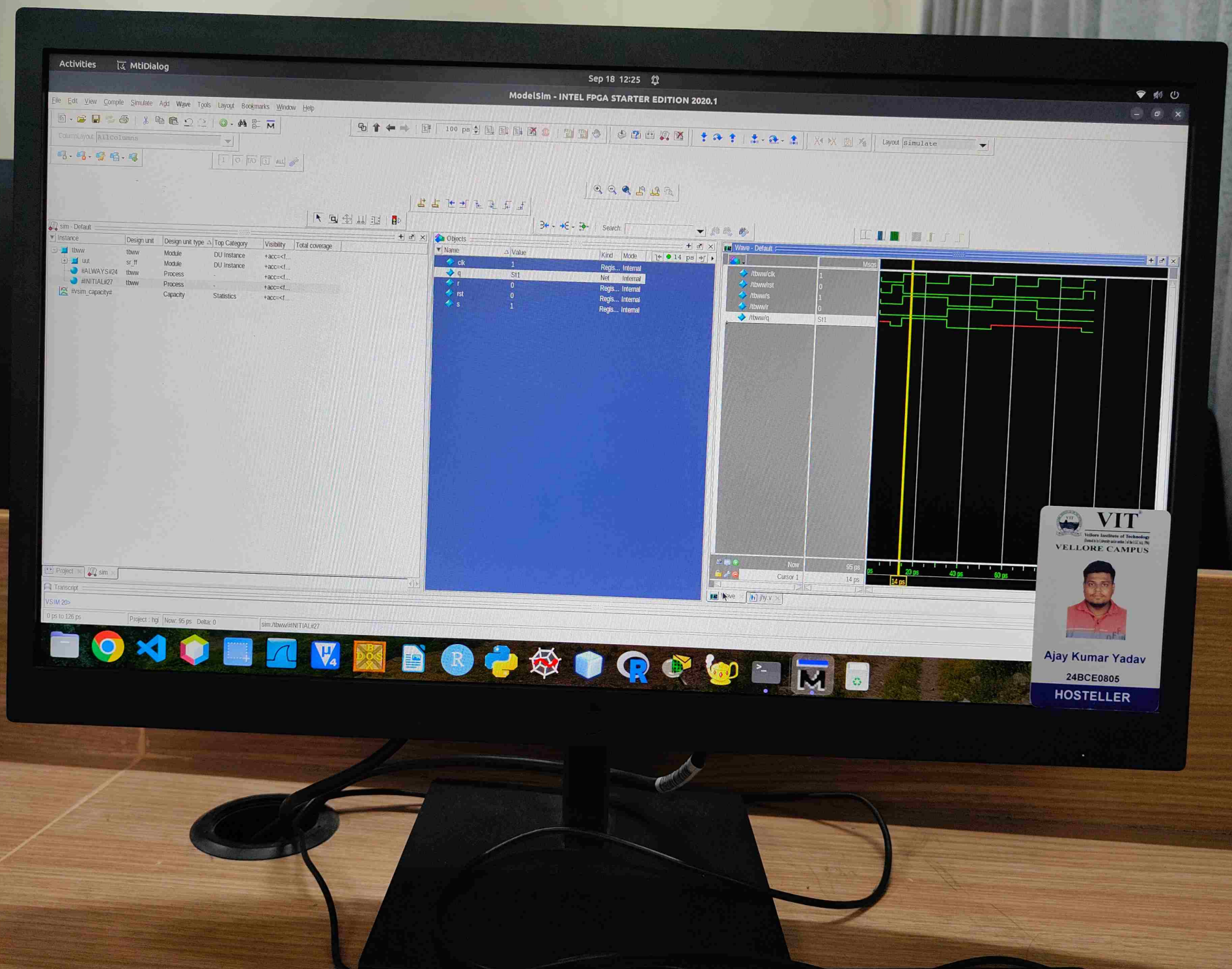
```
        clk <= 0; rst <= 1;
```

```
        # 10 rst <= 0;
```

```
        # 200 $stop;
```

```
    end
```

```
endmodule
```

Activities MTIDialog

ModelSim - INTEL FPGA STARTER EDITION 2020.1

File Edit View Compile Simulate Add Wave Tools Layout Bookmarks Window Help

Calculator

1 0 10 11 12

Search

Search

Instance	Design unit	Design unit type	Top Category	Visibility	Total coverage
tb_jk	tb_jk	Module	DU Instance	+acc=cl...	
jk_tf	jk_tf	Module	DU Instance	+acc=cl...	
#ALWAYS#24	tb_jk	Process	-	+acc=cl...	
#INITIAL#27	tb_jk	Process	-	+acc=cl...	
evsim_capacity#	Capacity	Statistics	-	+acc=cl...	

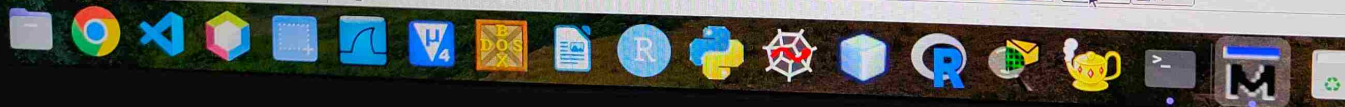
Name	Value	Kind	Mode
clk	1	Regis... Internal	
j	0	Regis... Internal	
k	1	Regis... Internal	
q	S10	Net Internal	
rst	0	Regis... Internal	



Project: hg

Now: 100 ps Delta: 2

sim/tb_jk



activities MtiDialog

Sep 18 1:00 PM

ModelSim - INTEL FPGA STARTER EDITION 2020.1

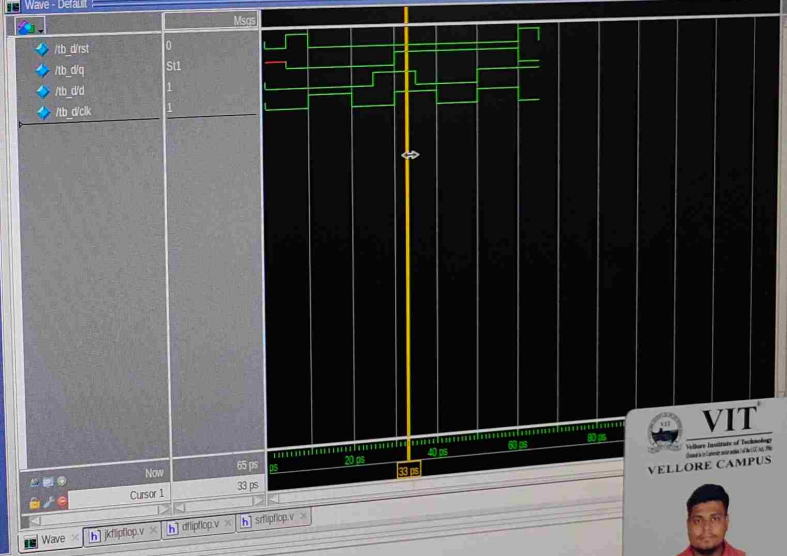
Edit View Compile Simulate Add Wave Tools Layout Bookmarks Window Help

Layout Simulate

ColumnLayout Default

Instance	Design unit	Design unit type	Top Category	Visibility
tb_d	tb_d	Module	DU Instance	+acc=<f
uut	d_ff	Module	DU Instance	+acc=<f
#ALWAYS#18	tb_d	Process	-	+acc=<f
#INITIAL#21	tb_d	Process	-	+acc=<f
#vsim_capacity#		Capacity	Statistics	+acc=<f

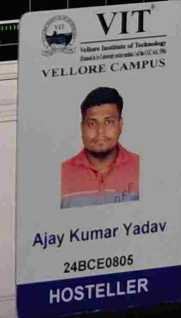
Name	Value	Kind	Mode
rst	0	Regi...	Internal
q	St1	Net	Internal
d	1	Regi...	Internal
clk	1	Regi...	Internal



```

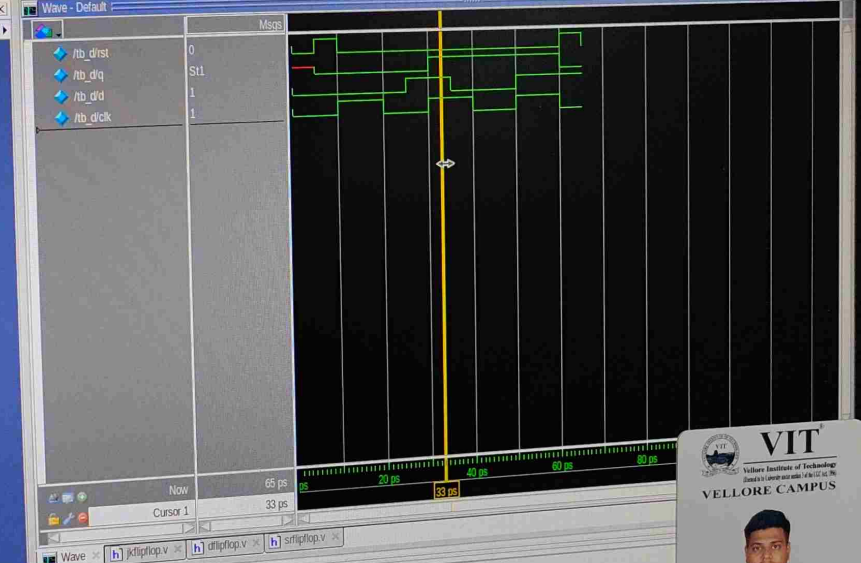
# Loading work.tb_d
# Loading work.d_ff
add wave -position insertpoint \
sim:/tb_d/rst \
sim:/tb_d/q \
sim:/tb_d/d \
sim:/tb_d/clk \
VSIM 34> run
# Note: Sstop : /home/matlab/downloads/24bce0837/dflplop.v(32)
# Time: 65 ps Iteration: 0 Instance: /tb_d
# Break in Module tb_d at /home/matlab/downloads/24bce0837/dflplop.v line 32
VSIM 35>
0 ps to 126 ps
Project: rush Now: 65 ps Delta: 0

```



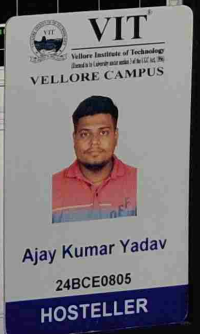
sim - Default	Design unit	Design unit type	Top Category	Visibility
Instance				
tb_d	tb_d	Module	DU Instance	+acc=<l
uut	d_ff	Module	DU Instance	+acc=<l
#ALWAYS#18	tb_d	Process	-	+acc=<l
#INITIAL#21	tb_d	Process	-	+acc=<l
#vsim_capacity#		Capacity	Statistics	+acc=<l

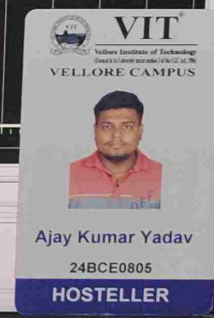
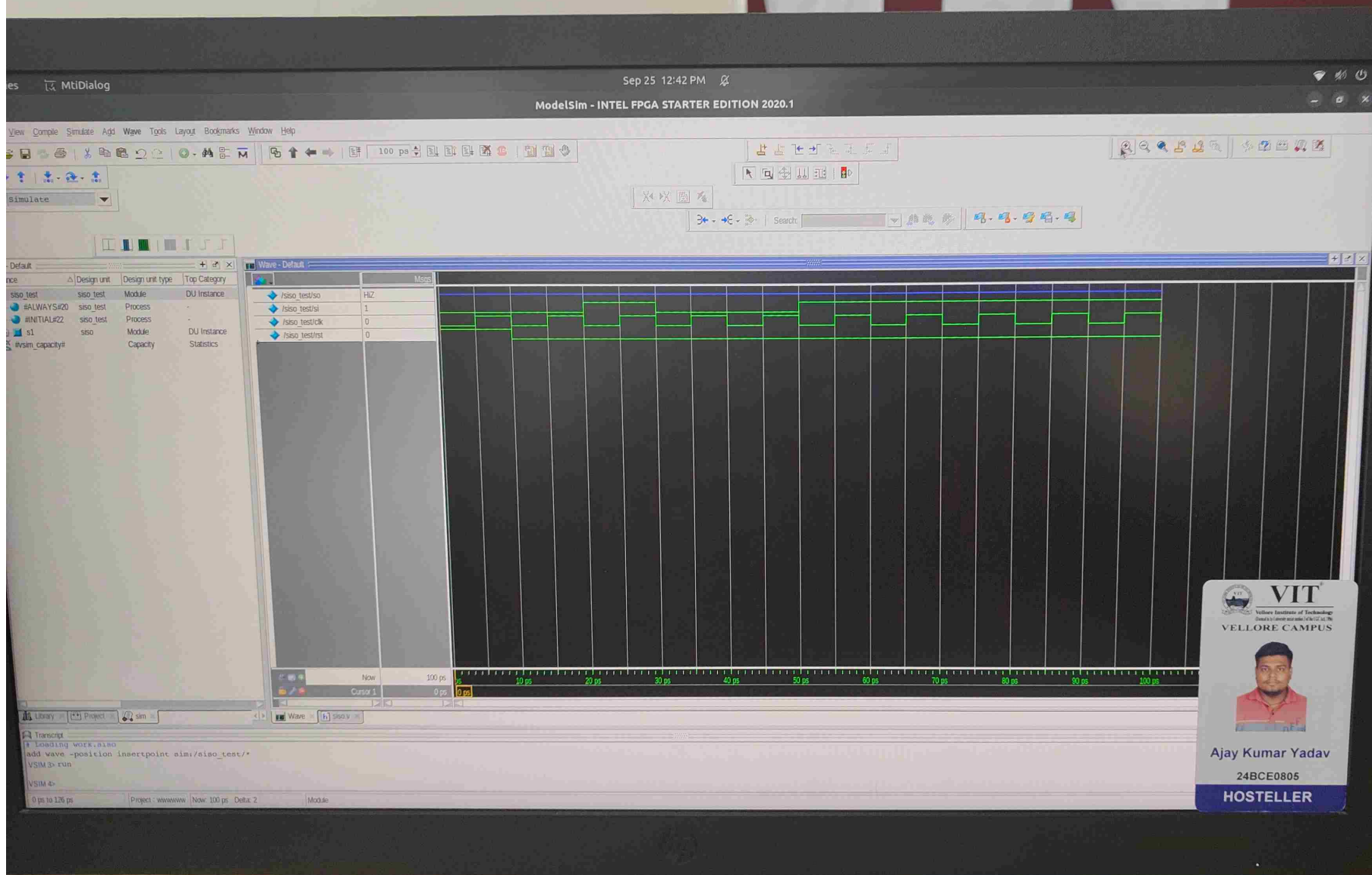
Objects	Name	Value	Kind	Mode
	rst	0	Regi...	Internal
	q	St1	Net	Internal
	d	1	Regi...	Internal
	clk	1	Regi...	Internal

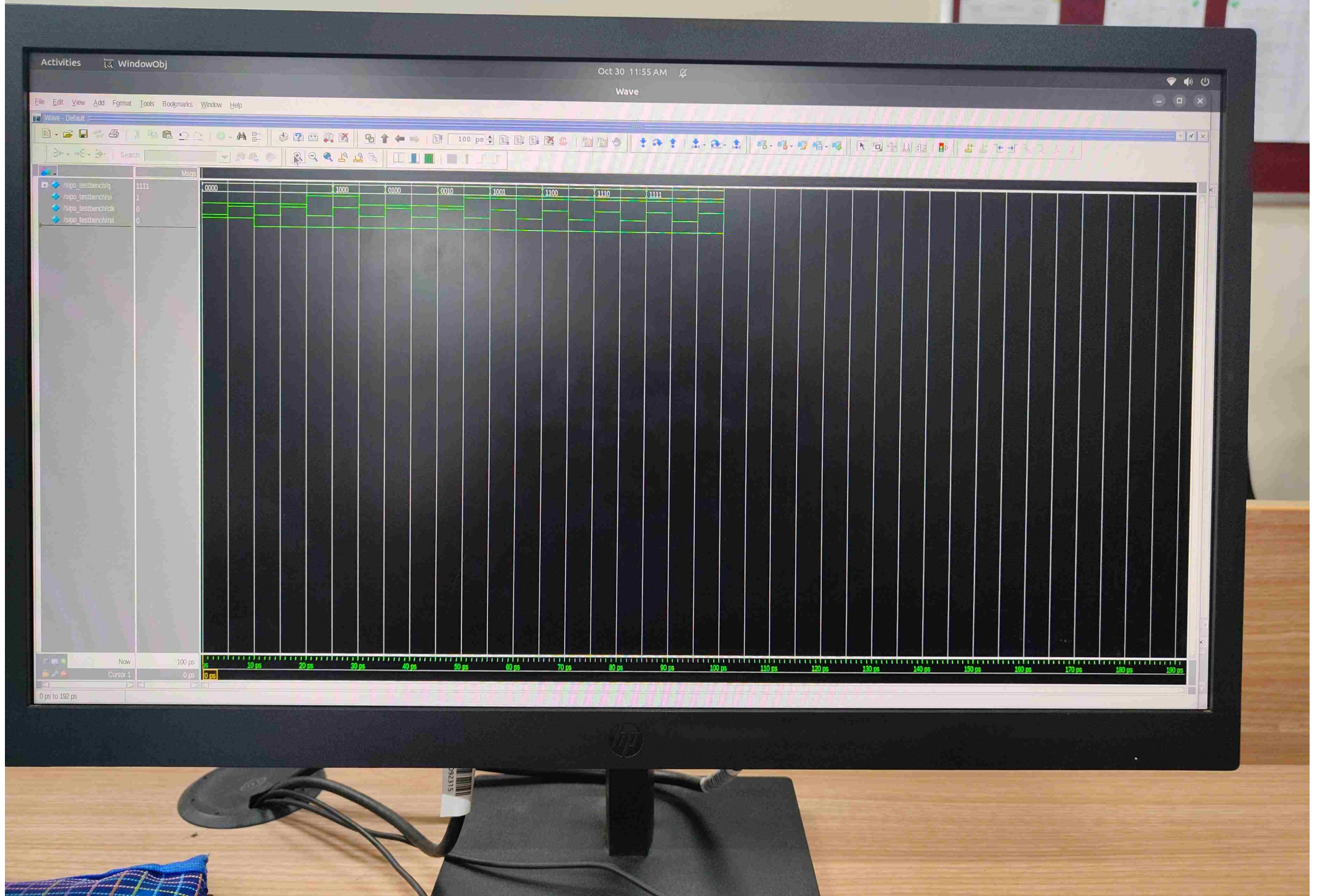


```
Transcript
# Loading work.tb_d
# Loading work.d ff
add wave -position insertpoint \
sim:/tb_d/rst \
sim:/tb_d/q \
sim:/tb_d/d \
sim:/tb_d/clk
VSI3M 34> run
** Note: Sstop : /home/matlab/downloads/24bce0837/dflipflop.v(32)
Time: 65 ps Iteration: 0 Instance: /tb_d
# Break in Module tb_d at /home/matlab/downloads/24bce0837/dflipflop.v line 32
VSI3M 35>
```

Project : rush Now: 65 ps Delta: 0 /tb_d/clk
0 ps to 126 ps







Activities

WindowObj

Oct 30 12:18 PM

Wave

File Edit View Add Format Tools Bookmarks Window Help

Wave - Default



Search:



Msgs

1001



/up_down_test/Q

1



/up_down_test/up_down

0



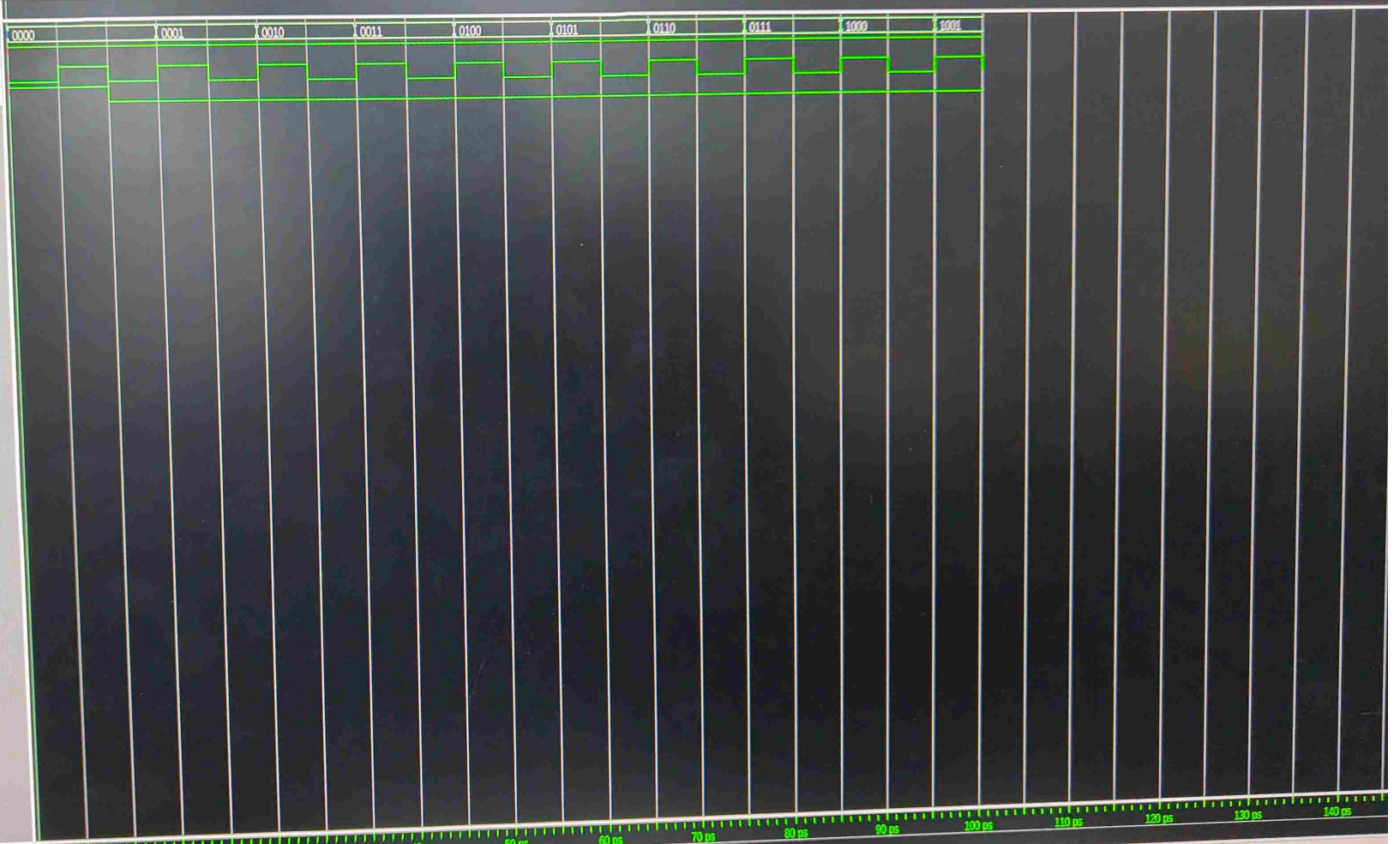
/up_down_test/clk

0



/up_down_test/rst

0

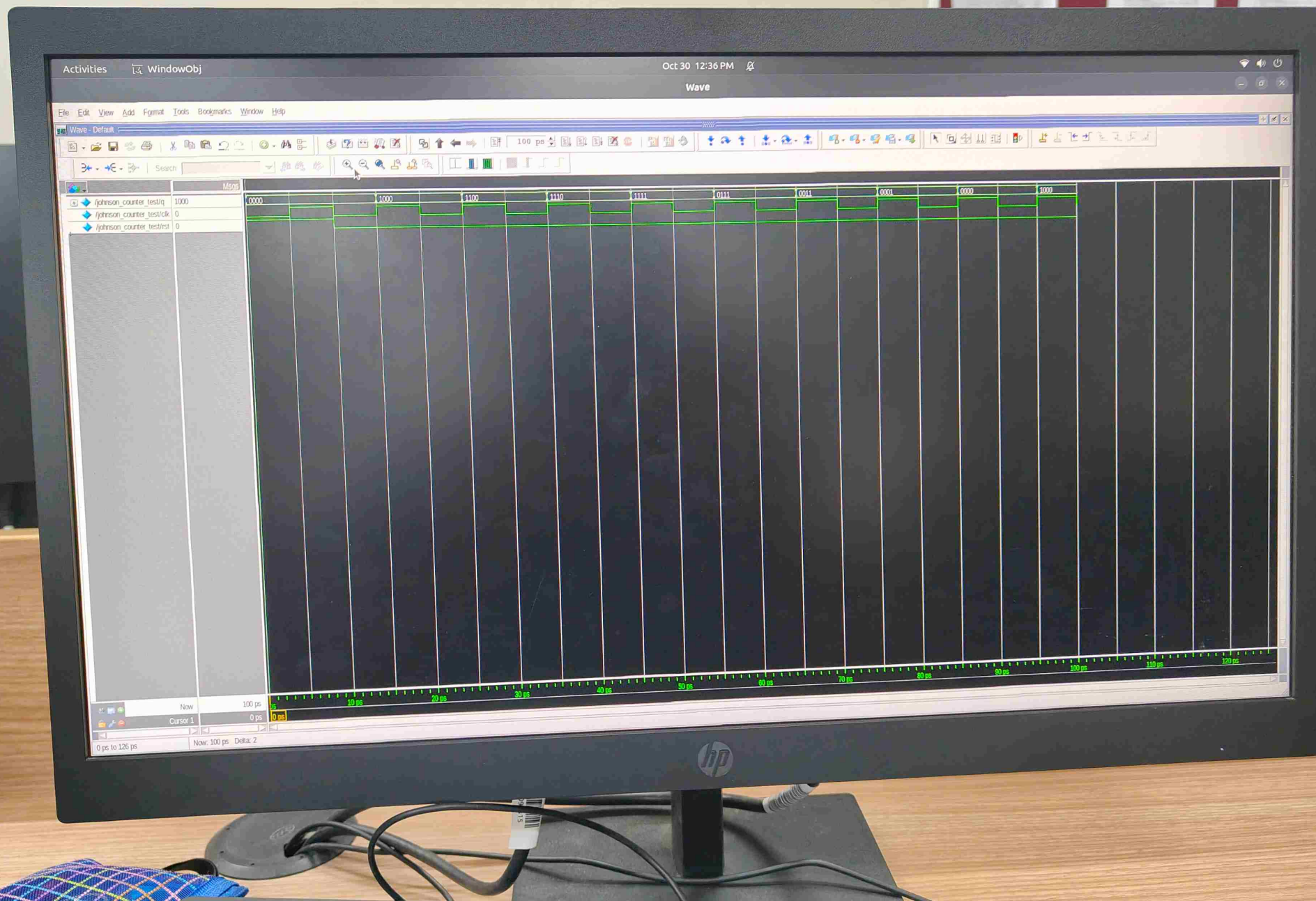


Now 100 ps

Cursor 1 0 ps

0 ps to 178 ps

Now: 100 ps Delta: 2



Activities WindowObj

Oct 30 12:57 PM

Wave

File Edit View Add Format Tools Bookmarks Window Help

Wave - Default

Search

	Msps
/seq_mealy_test/y	S10
/seq_mealy_test/i	0
/seq_mealy_test/clk	0
/seq_mealy_test/rst	0

